

[Download Firefox](#)

HACKS

🔍 Поиск Взломов Mozilla

За кулисами: укрепление Firefox с помощью Claude Mythos Preview



Авторы: [Брайан Гринстед](#), [Кристиан Холлер](#), [Фредерик Браун](#)

Опубликовано 7 мая 2026 г. в разделе [«Избранные статьи»](#), [Firefox](#), и [Безопасность](#)

Две недели назад мы [объявили](#), что с помощью Claude Mythos Preview и других моделей искусственного интеллекта нам удалось выявить и устранить беспрецедентное количество скрытых уязвимостей в Firefox. В этом посте мы подробнее расскажем о том, как мы подошли к этой работе, что мы обнаружили, а также дадим советы другим проектам о том, как эффективно использовать новые возможности для защиты от атак.

Внезапно оказалось, что баги — это очень даже хорошо

Всего несколько месяцев назад отчеты об уязвимостях в проектах с открытым исходным кодом, сгенерированные искусственным интеллектом, в основном воспринимались как нежелательный мусор. Работа с отчетами, которые кажутся правдоподобными, но на самом деле содержат ошибки, ложится асимметричным бременем на разработчиков проектов: легко и дешево заставить большую языковую модель найти «проблему» в коде, но реагировать на нее — долго и дорого.

Трудно переоценить, насколько сильно эта динамика изменилась для нас за несколько коротких месяцев. Это произошло благодаря сочетанию двух основных факторов. Во-первых, модели стали гораздо более функциональными. Во-вторых, мы значительно усовершенствовали наши методы *использования* этих моделей — управления ими, масштабирования и объединения в кластеры для получения большого количества сигнала и фильтрации шума.

Обычно мы не раскрываем подробные отчеты об ошибках в течение нескольких месяцев после выпуска исправлений и рекомендаций по безопасности, в основном из соображений предосторожности, чтобы защитить пользователей, которые по каким-либо причинам не спешили обновляться до последней версии Firefox. Учитывая чрезвычайный интерес к этой теме и необходимость срочных действий во всей экосистеме программного обеспечения, мы приняли взвешенное решение раскрыть небольшую часть отчетов, связанных с недавно выпущенными исправлениями. Мы постарались собрать их из разных подсистем браузера, но процесс отбора все равно был в некоторой степени произвольным. Тем не менее мы надеемся, что глубина и разнообразие этих докладов подтверждают нашу оценку возможностей и призыв к правозащитникам начать применять эти методы.

Идентификатор ошибки	Описание
2024918	Некорректная проверка на равенство может привести к тому, что JIT-компилятор оптимизирует инициализацию активной структуры WebAssembly GC, создавая примитивный «поддельный» объект с потенциальными возможностями произвольного чтения и записи в коде, который прошел тщательную фаззинг-проверку внутренними и внешними исследователями.
2024437	15-летняя ошибка в элементе <legend>, вызванная тщательной проработкой пограничных случаев в разных частях браузера, включая ограничения глубины стека рекурсии, свойства expando и циклическую сборку мусора.
2021894	Надежно использует состояние гонки при межпроцессном взаимодействии, позволяя скомпрометированному процессу с контентом манипулировать счетчиками ссылок IndexedDB в родительском процессе, чтобы вызвать UAF и потенциальный выход за пределы изолированной программной среды.
2022034	Необработанный NaN, пересекающий границу IPC, может маскироваться под указатель на объект JS с тегом, превращая двойную десериализацию в примитив родительского процесса для выхода из песочницы.
2024653	Сложный тестовый пример, в котором используются вложенные циклы обработки событий, обработчики скрытия страницы и сборка мусора для выявления уязвимости в установщике атрибутов для элементов <object>.
2022733	Запускает родительский UAF, заваливая WebTransport тысячами хэшей сертификатов, чтобы создать состояние гонки в цикле копирования с большим количеством ссылок, и использует это

Идентификатор ошибки	Описание
	состояние гонки для межпроцессного взаимодействия из скомпрометированного процесса.
2023958	Имитирует вредоносный DNS-сервер, перехватывая вызовы DNS-функций <code>glibc</code> , чтобы воспроизвести пограничный случай с переходом с UDP на TCP, вызывающий переполнение буфера и утечку памяти в стеке родительского процесса при разборе HTTPS RR и ECH.
2025977	Ошибка в XSLT, обнаруженная 20 лет назад, из-за которой реентерабельные вызовы <code>key()</code> приводили к перехешированию хэш-таблицы, что приводило к освобождению ее хранилища, пока еще использовался необработанный указатель на запись (это одна из нескольких серьезных проблем, связанных с XSLT, которые мы устранили).
2027298	Патч для палитры цветов, имитирующий выбор цвета пользователем, который в противном случае не мог бы быть автоматизирован. Затем с помощью синхронного события ввода запускается вложенный цикл обработки событий, который повторно запускает завершение работы актора и освобождает обратный вызов, пока он еще выполняется, запуская UAF для обработки контента.
2023817	Взломанный контентный процесс может отправить произвольное изображение обоев для декодирования в родительский процесс, что в сочетании с гипотетической уязвимостью в декодере изображений может привести к выходу из «песочницы». Это требовало сложных для автоматизации рассуждений об уровне доверия входных данных в родительском процессе.
2029813	Обходит нашу технологию изолированной программной среды для сторонних библиотек (RLBox), используя уязвимость в логике проверки, которая используется для копирования значений с недоверенной стороны изолированной программной среды на доверенную.
2026305	Крайне простой тестовый пример, использующий специальную семантику <code>rowspan=0</code> в HTML-таблицах. Он добавляет >65535 строк, чтобы обойти ограничение и переполнить 16-битное битовое поле макета, что годами оставалось незамеченным для фаззеров.

Note that a number of these bugs are *sandbox escapes*, which would need to be combined with other exploits to achieve a *full-chain* Firefox compromise. These reports presume that the sandboxed process that renders site content has already been compromised with some separate bug, and is now running attacker-controlled machine

code attempting to escalate control into the privileged parent process. When crafting a sandbox escape, the model is permitted to patch the Firefox source code, so long as the modified code is restricted to run only in the sandboxed process^[1]. Such bugs are notoriously difficult to find with fuzzing, and while we've had some success [developing new techniques](#) to close this gap, AI analysis provides much more comprehensive coverage of this critical surface.

Just as interesting as what the models found is what they *didn't* find — not because they didn't try, but because they were unable to circumvent Firefox's layered defenses. For example, in recent years we received several clever reports from security researchers that managed to escape the process sandbox by triggering prototype pollution in the privileged parent process. Rather than fixing these problems one-by-one, we made an [architectural change](#) to freeze these prototypes by default. While auditing logs from the harness, we saw many *attempts* to pursue this line of escape that were thwarted by this design. Observing such direct payoff from previous hardening work was even more rewarding than finding and fixing more bugs.

Harnessing Models to Build a Hardening Pipeline

We've experimented internally with LLM code audits over the past few years, with early attempts using models like GPT 4 or Sonnet 3.5 to statically analyze high risk code for vulnerabilities. These experiments showed some promise, but the high rate of false positives made them impractical to scale.

The introduction of agentic harnesses that can reliably detect security issues has completely changed this. These can find real bugs *and* dismiss unreproducible speculation. The key feature of such a harness is that, given the right interfaces and instructions, it can create and run reproducible test cases to dynamically test hypotheses about bugs in code. After fixing the initial set of issues that [Anthropic sent to us](#) in February, we built our own harness atop our [existing fuzzing infrastructure](#).

We began with small-scale experiments prompting the harness to look for sandbox escapes with Claude Opus 4.6. Even with this model, we identified an impressive amount of previously-unknown vulnerabilities which required complex reasoning over multiprocess browser engine code. At first, we supervised the process in the terminal to observe the process in real-time and tune the prompts and logic. Once this was working well, we parallelized the jobs across multiple ephemeral VMs, each tasked to hunt for bugs within a specific target file and write its findings back to a bucket.

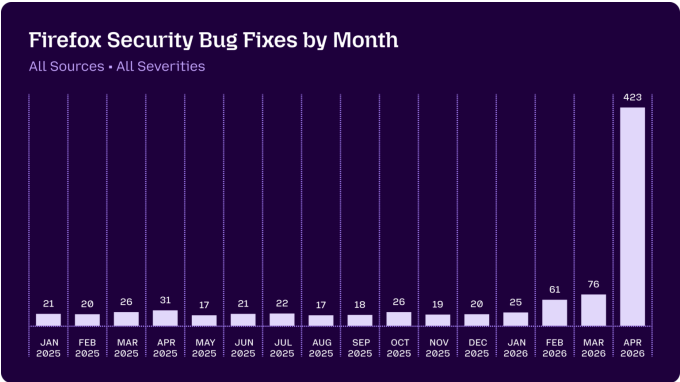
A discovery subsystem is necessary but not sufficient. In order to scale the effort, we needed to integrate it with our full security bug lifecycle: determining what to look for, where to look, and how to handle what it produces. This last part includes deduplicating against known issues, tracking bugs, triaging them, and getting fixes shipped. While the model is the core primitive powering the harness, this full pipeline is necessary to make it useful at scale.

While harnesses may be reusable across projects, this pipeline is inherently project-specific, reflecting each codebase’s semantics, tooling, and processes. Standing this up required significant iteration, with a tight feedback loop alongside the Firefox engineers who were fielding the incoming bugs.

Upgrading the Models

Once the end-to-end pipeline is in place, it’s trivial to swap in different models when they become available. Building this pipeline early helped us find a number of serious bugs using publicly-available models, and it also helped us hit the ground running when we had the opportunity to evaluate Claude Mythos Preview. In our experience, model upgrades increase the effectiveness of the entire pipeline: the system gets simultaneously better at finding potential bugs, creating proof-of-concept test cases to demonstrate them, and articulating their pathology and impact.

In addition to fixing the 271 bugs identified by Claude Mythos Preview in the [150 release](#), we’ve shipped more of these fixes in [149.0.2](#), [150.0.1](#), and [150.0.2](#). We also continue to find bugs with other means internally, and, similar to other projects, we’ve seen a significant uptick in external reports in the last few months.



Ultimately, every bug requires care and attention to properly fix. Staying on top of this unprecedented volume has led to a lot of work and long days over the last few months, and we’re extremely proud of how the team has stepped up to meet this challenge. Over 100 people contributed code to this effort to ship the most secure Firefox yet. In addition to writing and reviewing patches, others have been building and scaling this pipeline, triaging, testing the fixes, and managing the release process for each bug.

Takeaways

Anyone building software can start using a harness with a modern model to find bugs and harden their code today. We recommend getting started now. You will find bugs, and you will set yourself up to take advantage of new models as soon as they become available.

You can start with very simple prompting, then observe and iterate. Our initial prompts were not dissimilar from those described [here](#). Through iteration we've built out a lot of orchestration and tooling to optimize and scale the pipeline, but the essence of the inner loop remains the same: there is a bug in this part of the code, please find it and build a testcase.

We haven't bottomed on all the latent bugs in Firefox, but are quite pleased with the trajectory. Today, our scanning is largely focused on specific areas of the code (files, functions) where we instruct the system to look, based on a mix of human judgement and automated signals. In the near future, we intend to integrate this analysis into our continuous integration system to scan patches as they land in the tree. Models are quite flexible with the form of context provided, and we expect patch-based scanning to work as well or even better than file-based scanning.

The current moment is a perilous one, but also full of opportunity. Let's work together to secure the internet.

FAQ

The announcement said “271 bugs”, but I count something different. What's going on?

On the [advisories web page](#) we group all internally-reported bugs as “rollup” CVEs with multiple bugs underneath them. The web page is built from [yaml](#) in the [foundation-security-advisories](#) repo, the canonical location for our CVE assignments. While some browsers do not create CVE identifiers for internally-discovered issues at all, we provide this information in order to be as transparent as possible.

In Firefox 150, there were three internal rollups: CVE-2026-6784 (154 bugs), CVE-2026-6785 (55 bugs), and CVE-2026-6786 (107 bugs).

Astute readers will notice the number of bugs in those internal rollups adds up to 316, which is more than the 271 we announced finding with Claude Mythos Preview. That's because our security team hunts for new bugs every day by attacking Firefox with a combination of (a) fuzzing systems (b) manual inspection and (c) this new agentic pipeline across a variety of models.

We fixed a total of 423 security bugs in releases in April. In addition to the 271 bugs announced two weeks ago, there were 41 externally reported bugs, with the remaining 111 discovered internally and split roughly in third between:

1. Bugs found using this pipeline with Claude Mythos Preview but fixed in releases other than Firefox 150
2. Bugs found using this pipeline with other models
3. Bugs found with other techniques like fuzzing

Note that we also directly credited 3 CVEs to Anthropic separate from this latest effort (CVE-2026-6746, CVE-2026-6757, CVE-2026-6758). These were fixes for bugs [sent to us](#) by the outstanding Anthropic Frontier Red team a couple months ago and we assigned unique CVEs for each as per our normal process.

What do security ratings mean?

As additional context, we apply [security severity ratings](#) from critical to low to indicate the urgency of a bug:

- sec-critical and sec-high are assigned to vulnerabilities that can be triggered with normal user behavior, like browsing to a web page. We make no *technical* difference between these, but sec-critical bugs are reserved for issues that are publicly disclosed or known to be exploited in the wild.
- sec-moderate is assigned to vulnerabilities that would otherwise be rated sec-high but require unusual and complex steps from the victim.
- sec-low is assigned to bugs that are annoying but far from causing user harm (e.g, a safe crash).

Of the 271 bugs we announced for Firefox 150: 180 were sec-high, 80 were sec-moderate, and 11 were sec-low.

While we care most about critical/high bugs, it's normal for us to prioritize moderate and low security bugs in order to fix correctness issues and as a defense-in-depth mechanism.

Is a sec-high or sec-critical bug the same as a practical exploit?

Not necessarily.

In most cases, a single critical/high bug is not actually enough to compromise Firefox. This is because Firefox has a defense-in-depth architecture, so for example exploiting a JIT bug only achieves remote code execution in a sandboxed and site-specific process. Real-world attackers generally need to chain multiple exploits together to escalate privileges through one or more layers of sandboxing along with OS-level mitigations like ASLR.

We also generally don't build exploits to see whether a bug could be used by an attacker in the real world. We classify sec-high based on predictable crash symptoms such as use-after-free or out-of-bounds memory issues being reported by AddressSanitizer, and our threat model assumes that any of them could be exploitable

with sufficient effort. This reduces the risk of a false negative during exploitability analysis, and more importantly it allows us to focus our resources on finding and fixing more vulnerabilities.

[1] Our bug bounty program has [similar rules](#). ↩

About Brian Grinstead

Distinguished Engineer, Firefox

[More articles by Brian Grinstead...](#)

About Christian Holler

Christian is a Firefox Tech Lead and Principal Engineer at Mozilla.

 [@mozdeco](#)

[More articles by Christian Holler...](#)

About Frederik Braun

Frederik Braun manages the Firefox Application Security team. He builds security for the web and for Mozilla Firefox from Berlin. As a contributor to standards, Frederik is also improving the web platform by bringing security into the defaults with specifications like the Sanitizer API and Subresource Integrity. When not at work, Frederik likes reading a good novel or going on long bike treks across Europe.

 <https://frederikbraun.de>

[More articles by Frederik Braun...](#)

Discover great resources for web development

Sign up for the Mozilla Developer Newsletter:

☐ I'm okay with Mozilla handling my info as explained in this [Privacy Policy](#).

[Sign up now](#)

Except where otherwise noted, content on this site is licensed under the Creative Commons Attribution Share-Alike License v3.0 or any later version.